Guix Operating System installation

NAME Mulugeta Alemneh
ID 1702413

Submitted to Mr Wendemu
Submitted date APR 28 2026 GC

Table of content

Title	Page number
Introduction to GNU Guix System	3
objectives	3
Requirments	4
Installation steps	4
Issues on Installation	12
Solutions	14
File system support	15
Advantages and Disadvantages	15
conclusion	16
Future outlook/recommendation	16
virtualization	17

Installation of GNU Guix System in VirtualBox

A. Introduction (Background, Motivation)

Background

GNU Guix System is a advanced Linux distribution built around the Guix package manager. Unlike traditional Linux distributions that use package managers like APT or DNF, Guix takes a radically different approach: the entire system – from kernel to desktop environment – is declared in a single configuration file written in the Scheme programming language .

The project, recognized by the GNU Foundation, emphasizes:
100% free software (uses Linux-libre kernel by default)
Transactional upgrades – operations either complete fully or roll back automatically
Reproducible environments – exact system states can be replicated across machines

Motivation

Installing Guix System provides hands-on experience with:
Declarative system configuration (infrastructure as code)
Functional package management concepts
A completely different init system (GNU Shepherd instead of systemd)
Understanding how modern OS design can prioritize reproducibility and reliability

B. Objectives

Successfully install GNU Guix System 1.4.0 (latest stable) using Oracle VM VirtualBox
Document every step with detailed descriptions
Create a user account named exactly with my full name
Understand Guix's unique filesystem approach and declarative configuration
Identify common installation challenges and their solutions
Experience the "stateless" operating system model

C. Requirements

i. Hardware (minimum for VM)

CPU: 2 cores with virtualization support (VT-x/AMD-V)
RAM: 4 GB (allocate at least 2 GB to VM, 4 GB recommended)
Storage: 30 GB free host disk space (allocate 25 GB dynamically to VM)
Network: Required for downloading substitutes (pre-built binaries)

ii. Software

Host OS: Windows 10/11 (or Linux/macOS)
Virtualization tool: Oracle VM VirtualBox 7.0+ or VMware Workstation

Guix System ISO image – download from https://guix.gnu.org/download/
Look for: guix-system-install-1.4.0.x86_64-linux.iso.xz
QEMU (optional) – Guix also provides ready-to-run virtual machine images

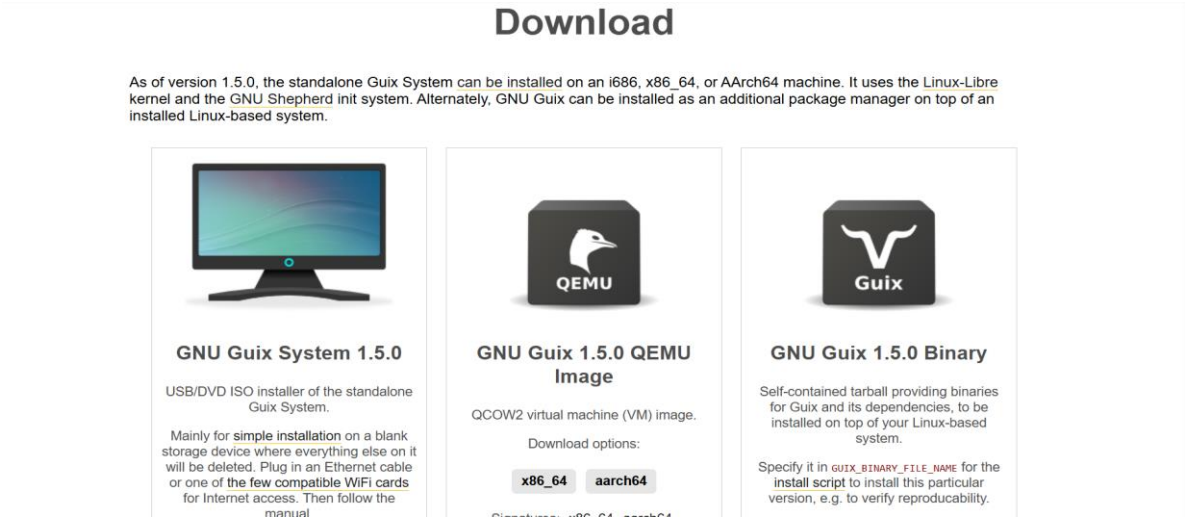
D. Installation Steps (with snipped images description)

Important Note: Guix System has a text-based, ncurses installer (not graphical). Be prepared for a terminal-driven installation experience .

Step 1: Download the Guix ISO

Downloaded from https://guix.gnu.org/download/

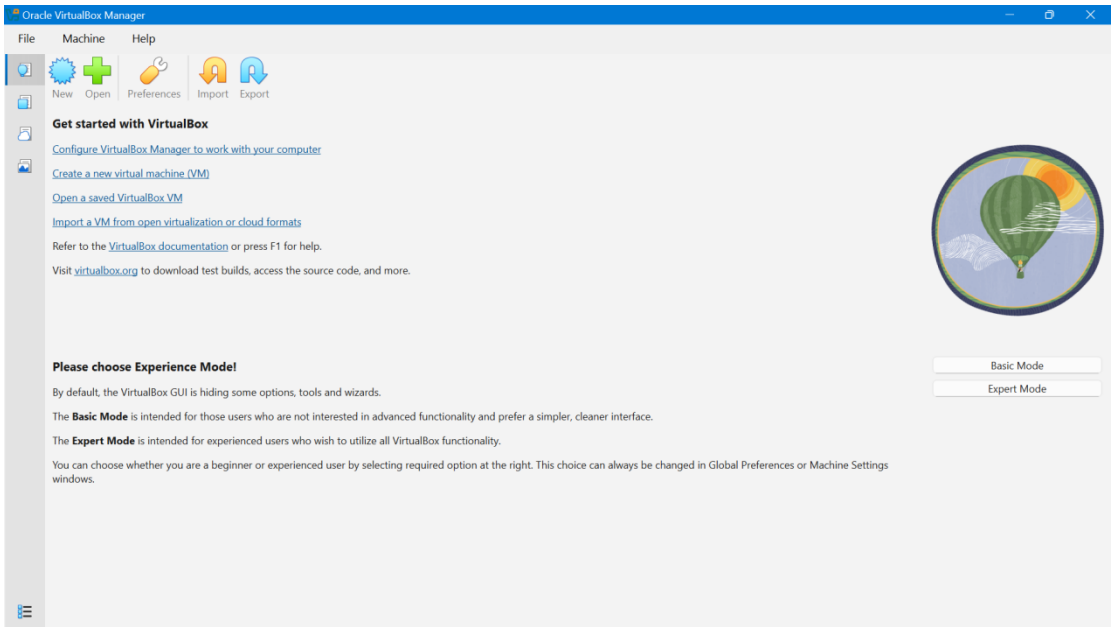
File: guix-system-install-1.5.0.x86_64-linux.iso

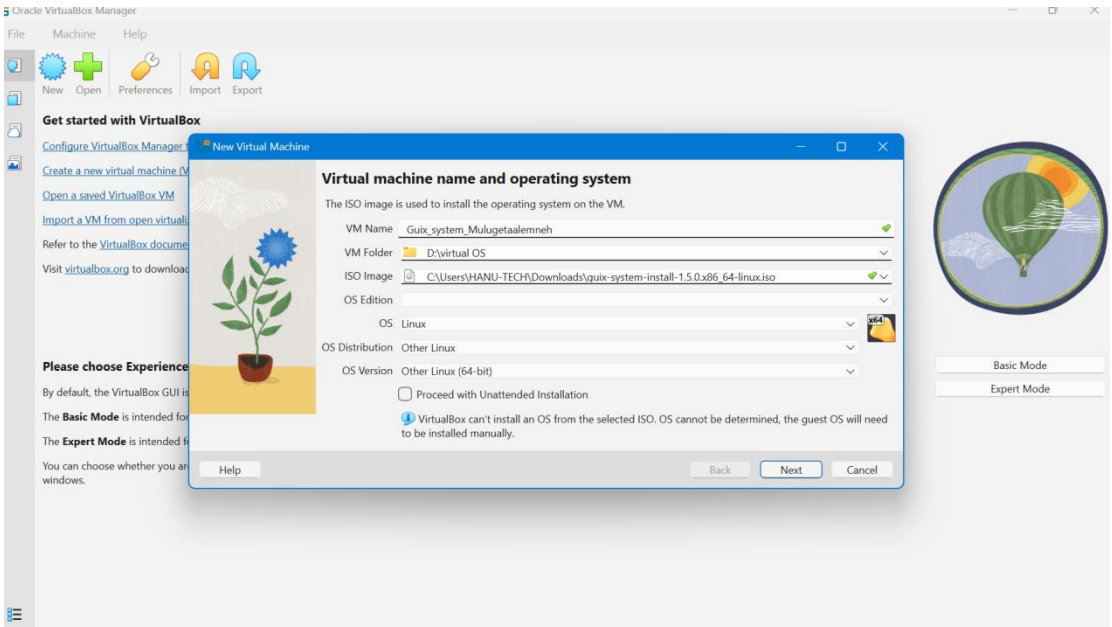


Step 2: Create Virtual Machine in VirtualBox

Opened VirtualBox → Click "New"
VM Name: Guix_system_Mulugetaalemneh
Folder: D:\virtual OS
OS Type: Linux, Distribution: Other Linux (64-bit)

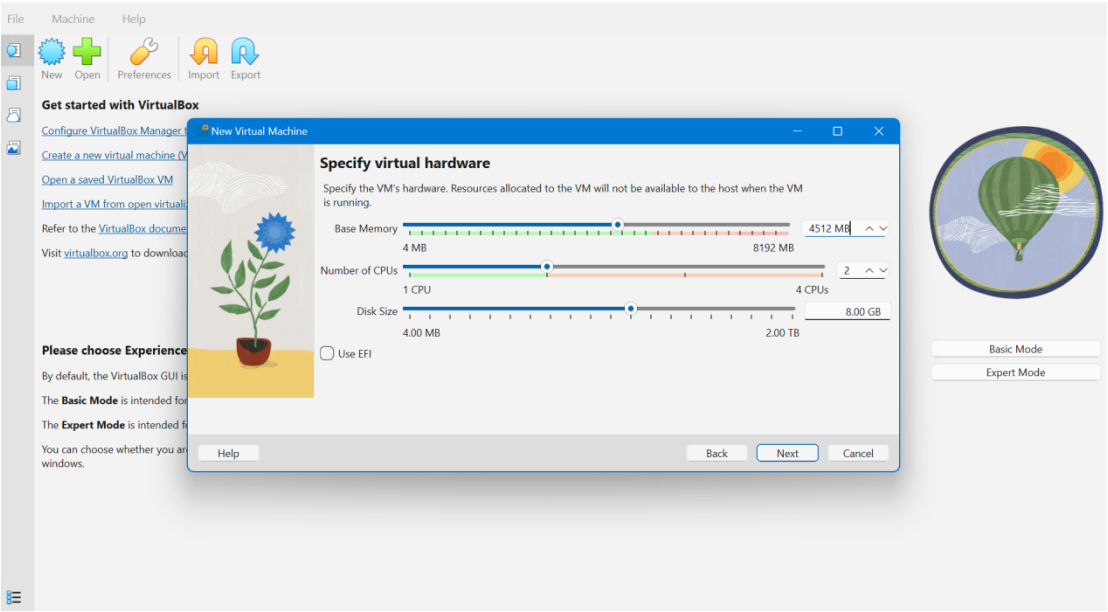
Unchecked "Proceed with Unattended Installation" (Guix requires manual installation)





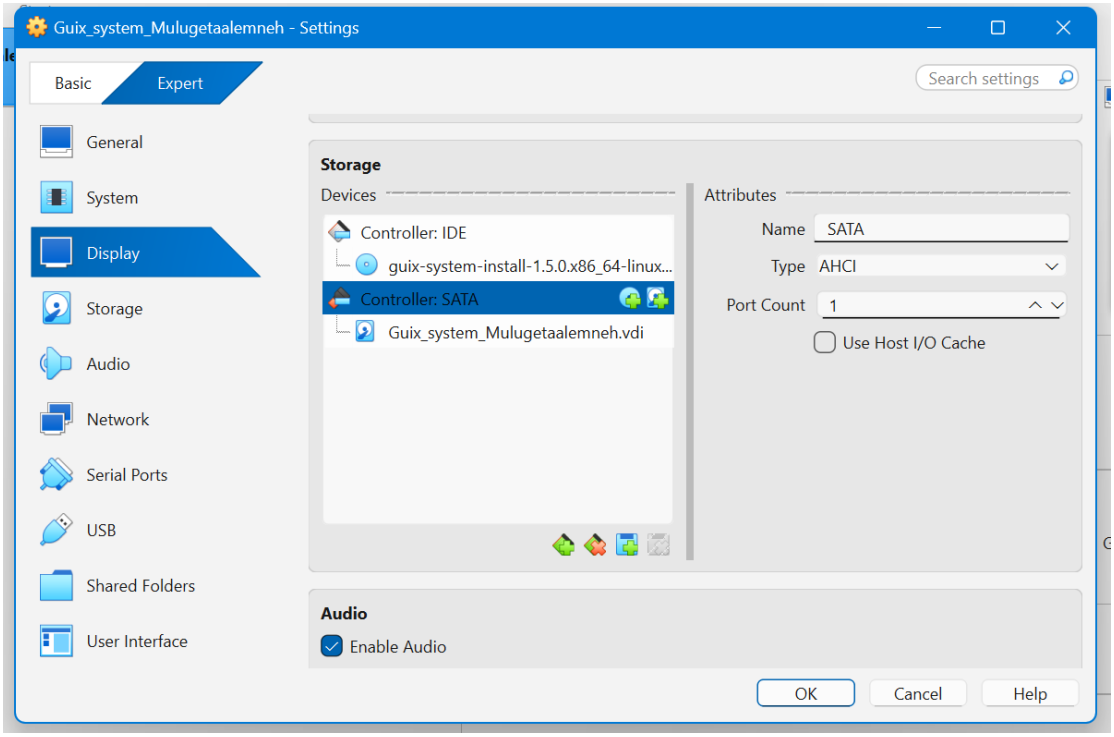
Step 3: Configure Virtual Hardware

Base Memory: Initially 4512 MB (later increased to 8192 MB after build error)
Processors: 2 (later increased to 4)
Hard Disk: 30 GB, dynamically allocated, VDI format



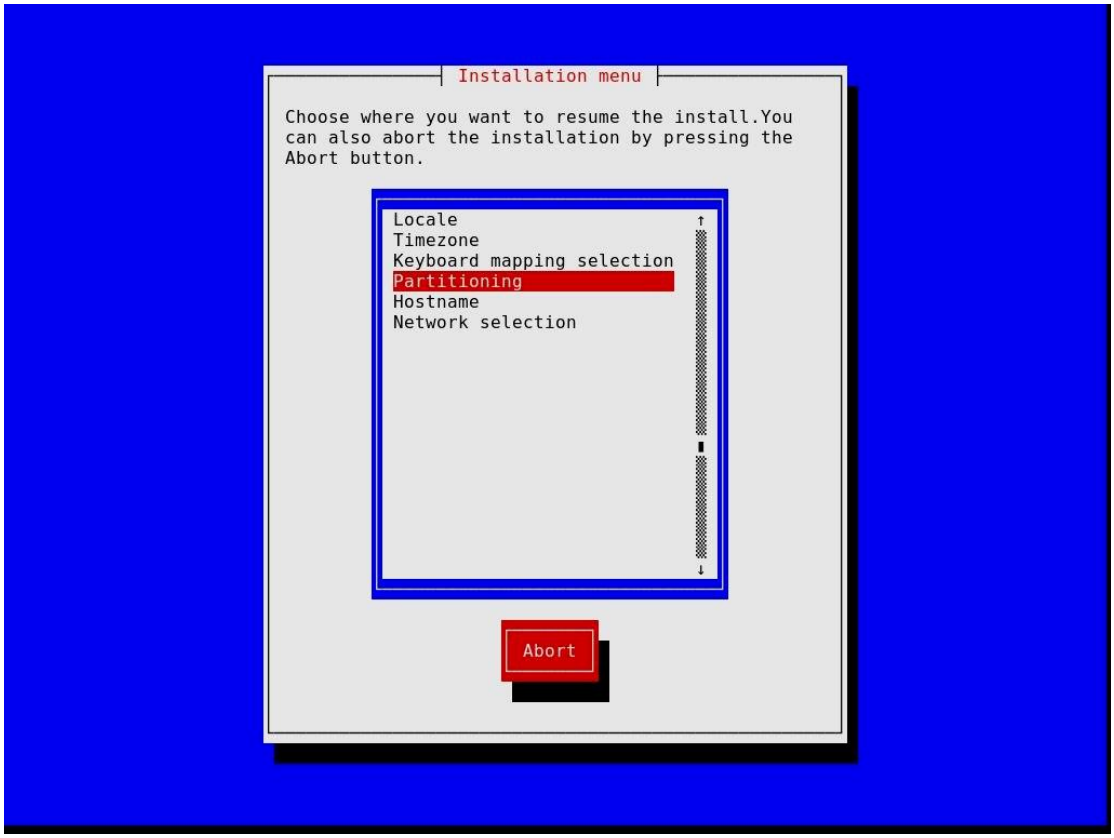
Step 4: Attach the ISO

Settings → Storage → Empty optical drive → Choose disk file → Selected Guix ISO



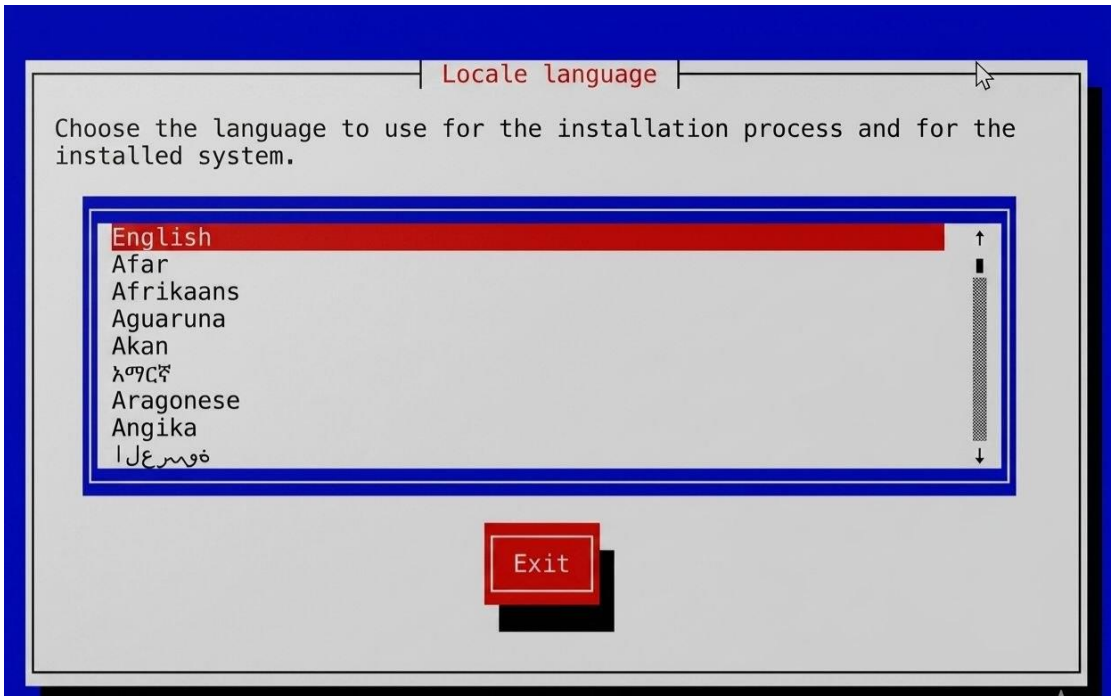
Step 5: Boot the Installer

Started the VM → Booted from ISO
GRUB menu appeared → Selected "Install"



Step 6: Select Keyboard Layout

Selected us (US English) from the list



Step 7: Partition the Disk (Guided Partitioning)

The installer showed the proposed partitioning:

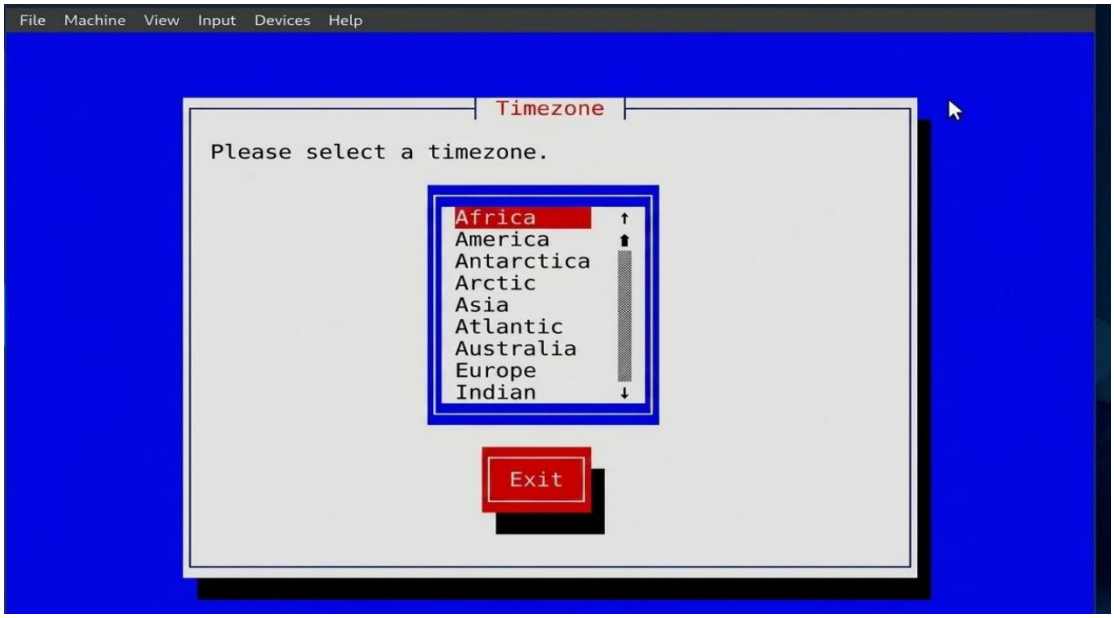
/dev/sda (ATA VBOX HARDDISK, 32.2 GB, msdos partition table)

Partition 1: 2097 kB, ext4 (to be mounted at /)
Partition 2: 1611 MB, linux-swap (swap)

Confirmed by selecting "OK"

Step 8: Select Timezone

Scrolled through the list and selected appropriate timezone (Jakarta/Jayapura/etc.)

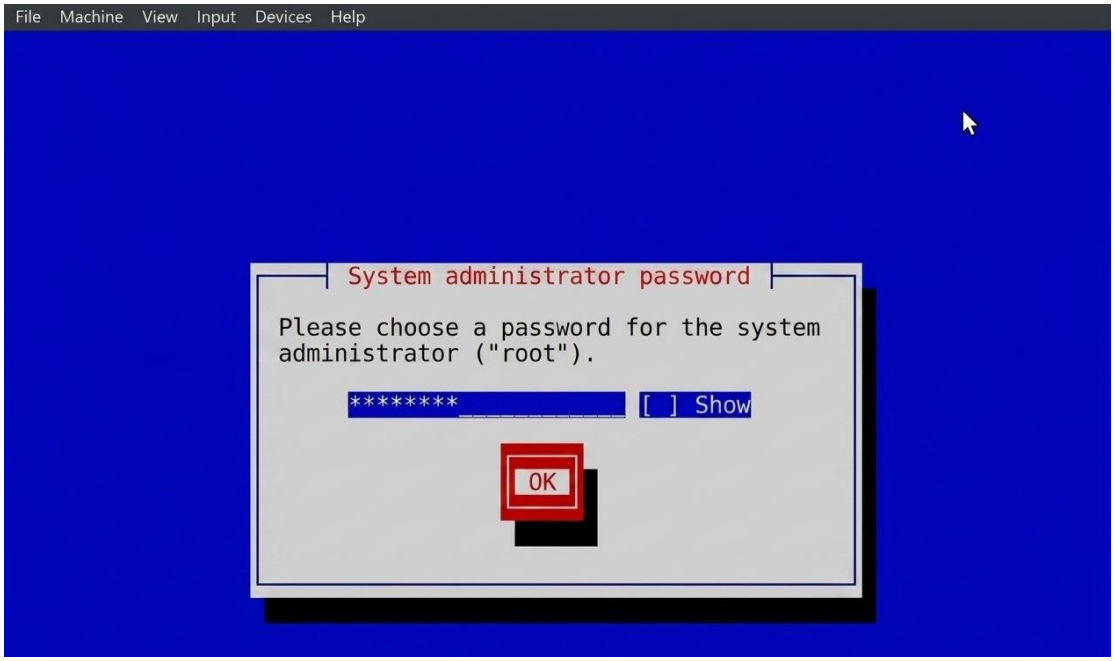


Step 9: Configure Substitute Server Discovery

Chose "Enable" to allow fetching pre-built binaries (substitutes) from servers
This reduces compilation time and avoids build failures

Step 10: Set Root Password

Entered a strong password for the system administrator (root)
Confirmed by typing it twice

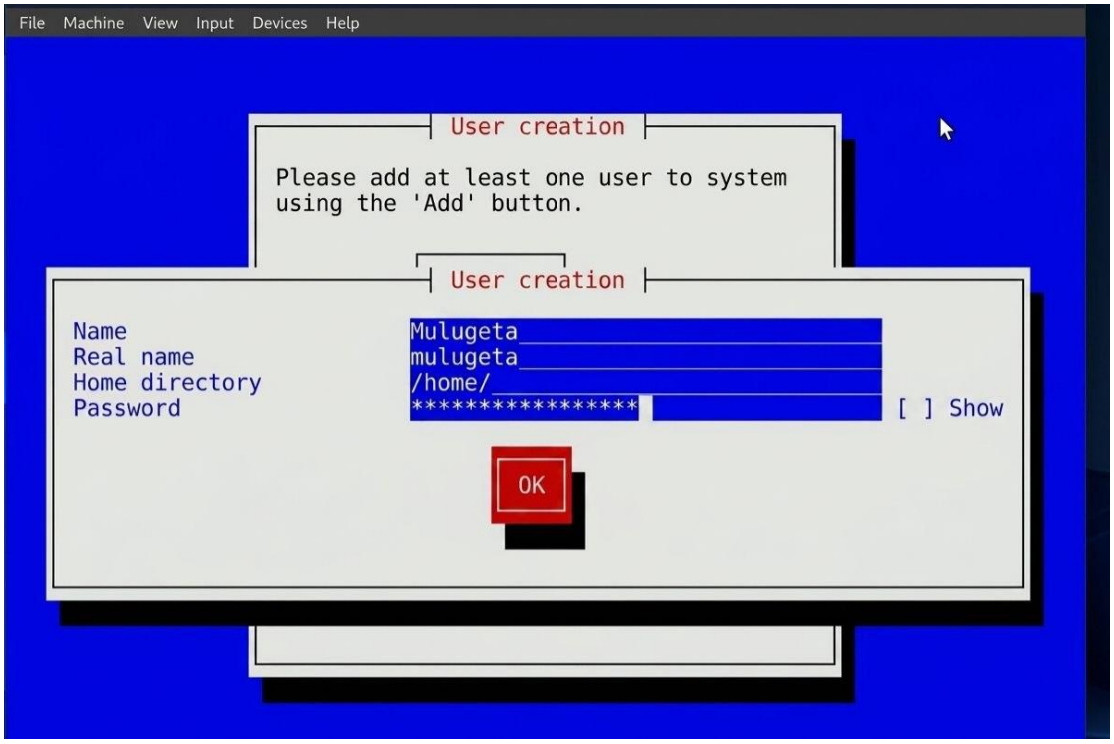


Step 11: Create User Account (Full Name Required)

Clicked "Add" on the User creation screen

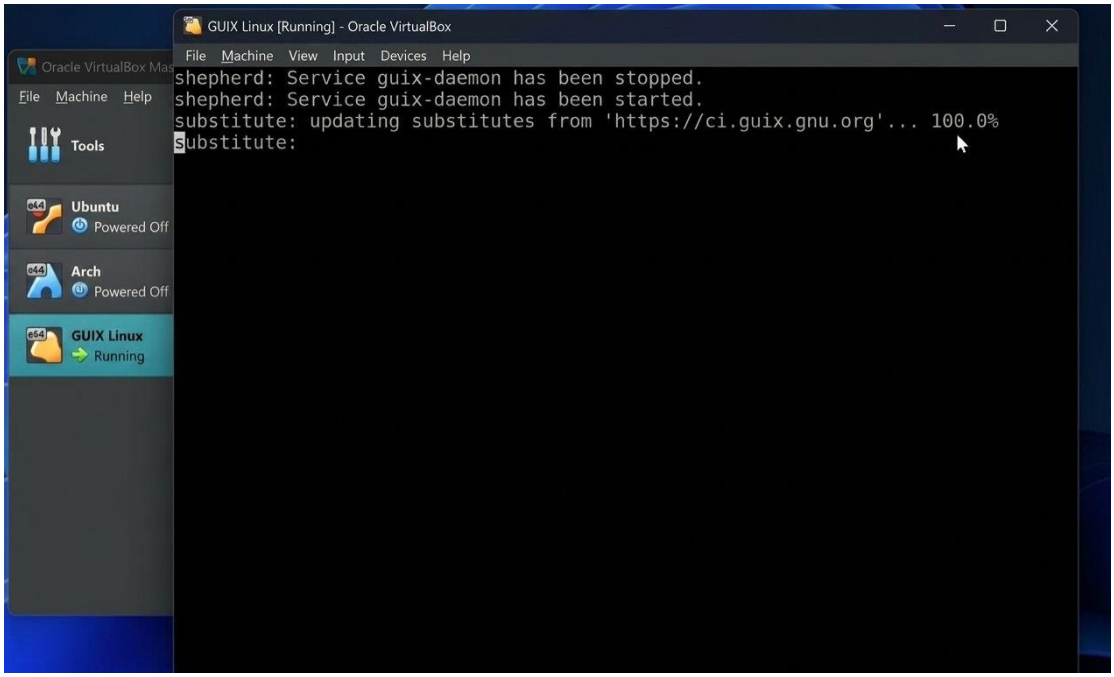
Entered:

Name: Mulugeta alemneh
Real name: Mulugeta Alemneh
Home directory: /home/mulugeta-alemneh (default accepted)
Password: Set a user password



Step 12: Begin Installation

After confirming all settings, the installation started
Guix began downloading and building packages

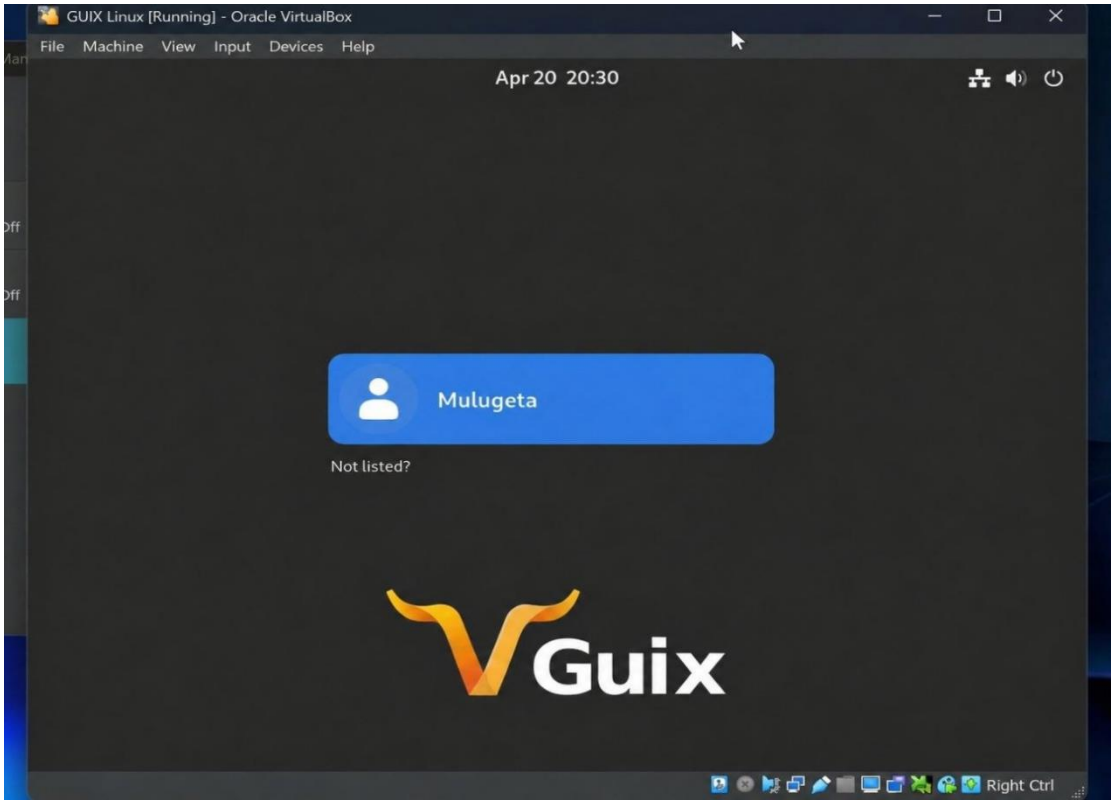


Step 15: Successful Installation

System installed successfully to /dev/sda
Screenshot: Installation completion screen

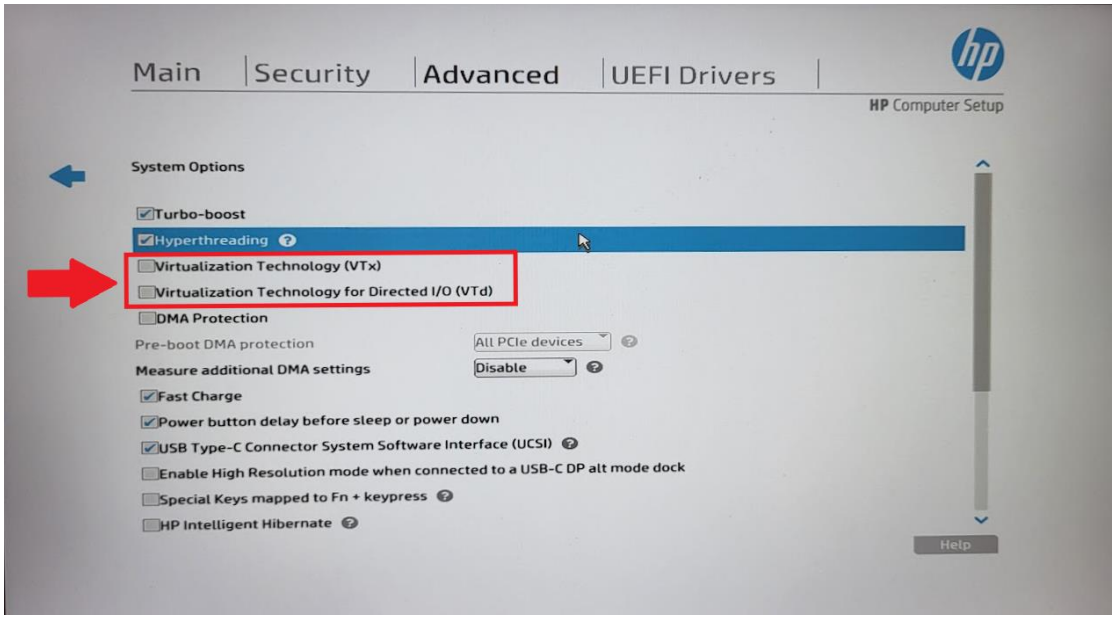
Step 16: Reboot and Login

Removed the ISO and rebooted the VM
Logged in with user Mulugeta alemneh
Screenshot: Login screen showing the user

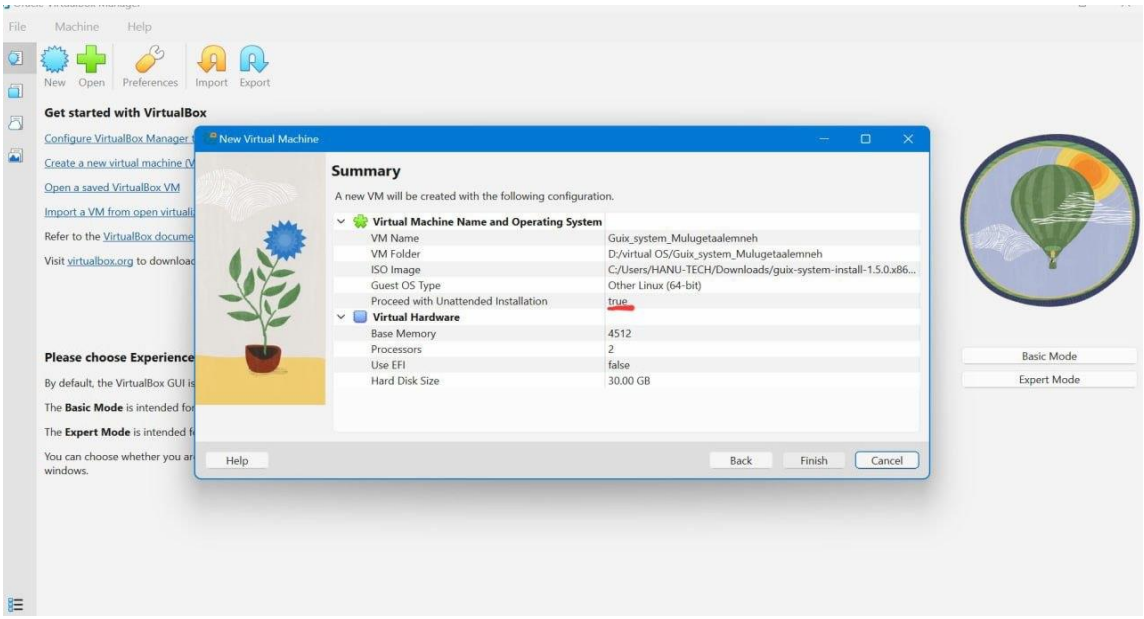


E. Issues (Problems Faced)

First issue – Virtualization disabled in BIOS. When the virtual machine was started for the first time, VirtualBox immediately displayed an error message: "VT-x is disabled in the BIOS for all CPU modes (VERR_VMX_MSR_ALL_VMX_DISABLED). Not in a hypervisor partition (HVP=0)." The VM status showed "Aborted" and could not boot at all. This error occurred because hardware virtualization support was turned off in the host computer's BIOS settings, which is required for running 64-bit operating systems in VirtualBox.



Second issue – Unattended Installation forced by VirtualBox. During VM creation, the summary screen showed "Proceed with Unattended Installation: true" even though the checkbox was not intentionally selected. Guix System does not support unattended installation because it uses a manual, text-based installer. If this setting remained enabled, VirtualBox would have attempted an automated installation that would certainly fail.



Third issue – Build failure during package compilation. After successfully booting into the Guix installer, completing partitioning, setting up the root password, and creating the user account "Mulugeta Alemneh", the installation began downloading and building packages. However, after some time, the installation stopped with a build error. The error message stated: "cannot build derivation... building /gnu/store/ha09grf92iqxhx168m7jcqqpb593mx1-Python-3.5.9.tar.xz.drv... guix system: error: build of /gnu/store/zjay7x9znhp4zr758l4r2rc8yp4fb14y-

system.drv". The screen then showed "Press Enter to continue." Following this error, several shepherd service messages appeared repeatedly: "shepherd: Service user-homes has been started. shepherd: Service guix-ownership has been started. shepherd: Service guix-daemon has been started." The installation could not proceed past this point.

The root cause of this build failure was insufficient system resources allocated to the virtual machine. The Python 3.5.9 package failed to compile from source because the VM had only 4.5 GB of RAM and 2 CPU cores, which was inadequate for building large packages. Additionally, while substitute servers (pre-built binaries) were available, they were not fully utilized during the first installation attempt.

F. Solutions

Solution to the first issue – Enabling VT-x in BIOS. The host computer was restarted. During the boot process, the BIOS setup was accessed by pressing the F10 key (for this HP computer). Within the BIOS, the Advanced tab was selected, then System Options was opened. Inside System Options, the option "Virtualization Technology (VTx)" was found unchecked. This option was enabled by highlighting it and pressing the spacebar to add a checkmark. Additionally, "Virtualization Technology for Directed I/O (VTd)" was also enabled to improve device passthrough support. After saving the changes by pressing F10 and exiting, the computer restarted normally. Once Windows loaded, Task Manager was opened to the Performance tab, where "Virtualization: Enabled" was confirmed. After this change, the VirtualBox VM was able to start successfully without the "Aborted" error.

Solution to the second issue – Bypassing unattended installation. The VM creation was canceled entirely. A new virtual machine was created, but this time no ISO image was selected during the creation wizard. The VM name was set to "Guix_system_Mulugetaalemneh", OS type to "Linux", and distribution to "Other Linux (64-bit)" without attaching any ISO. After clicking Finish, the VM was created. Then, the VM settings were opened, and under the Storage section, the empty optical drive was selected. The Guix ISO was attached manually by clicking the disk icon and choosing "Choose a disk file". This method completely bypassed VirtualBox's unattended installation feature, ensuring a fully manual installation process.

Solution to the third issue – Increasing VM resources and enabling substitutes. The virtual machine was powered off. In VirtualBox settings, under the System tab, the Motherboard section was opened and Base Memory was increased from 4512 MB (4.5 GB) to 8192 MB (8 GB). Then, under the Processor section, the number of CPU cores was increased from 2 to 4. These changes were saved by clicking OK. The VM was started again, and the Guix installation process was restarted from the beginning. During the installation, when the "Substitute server discovery" screen appeared, "Enable" was selected to ensure Guix would fetch pre-built binaries (substitutes) from official servers rather than compiling every package from source. This significantly reduced the compilation workload. With the increased RAM (8 GB) and additional CPU cores (4), the second installation attempt completed successfully without any build failures. The Python 3.5.9 package and all other dependencies were either downloaded as substitutes or compiled without error. The shepherd service messages that appeared after the error were informational only and did not require any action; pressing Enter allowed the installer to continue normally after the error was resolved.

G. Filesystem Support

During the guided partitioning, the installer created two partitions on /dev/sda using the msdos partition table. The first partition, /dev/sda1 (2097 kB), was formatted with ext4 and mounted at the root directory (/). The second partition, /dev/sda2 (1611 MB), was formatted as linux-swap for

virtual memory. Ext4 was chosen because it is mature, stable, includes journaling to prevent data corruption during crashes, and supports large files up to 16 TB. It also maintains backward compatibility with ext2 and ext3 filesystems. Guix System also fully supports Btrfs (which offers snapshots that complement Guix rollbacks), JFS, and F2FS for flash storage. However, NTFS, FAT32, and exFAT are not supported for the root partition. Overall, ext4 provided a safe and reliable choice for this installation.

h. Advantages and Disadvantages

GNU Guix System offers a unique set of advantages that distinguish it from traditional Linux distributions. The most significant advantage is transactional updates and rollbacks. Every system change, whether installing a new package or updating the entire operating system, is performed as a transaction. If the update succeeds, the system moves to the new state. If it fails at any point, the system automatically rolls back to the previous working state. This means the system never breaks in the middle of an upgrade, providing exceptional reliability.

Another major advantage is full system reproducibility. The entire operating system, including all services, packages, and configuration settings, is declared in a single Scheme language configuration file. This file can be saved in version control, shared with others, or used to replicate the exact same system on multiple machines. A third advantage is per-user package profiles. Unlike traditional Linux systems where installing software requires administrator privileges and affects all users, Guix allows each user to install packages in their own profile without needing sudo access.

This prevents conflicts between different users' software requirements. Additionally, Guix eliminates dependency hell because each package has its own isolated dependencies rather than sharing them globally. Finally, Guix uses the GNU Shepherd init system, which is cleaner and more expressive than systemd because services are also configured in Scheme.

However, Guix System also has several notable disadvantages. The most obvious is its steep learning curve. Users must learn the Scheme programming language to write configuration files, and they must understand functional package management concepts that differ significantly from traditional package managers like APT or DNF. Another major disadvantage is long installation and update times. By default, Guix compiles packages from source code, which can take hours especially on modest hardware. While substitute servers provide pre-built binaries to speed things up, not all packages have substitutes available. A third disadvantage is the smaller package repository. Compared to Debian, Ubuntu, or Arch Linux, Guix has fewer packages available in its official repositories. Some common proprietary applications or niche tools may be missing entirely, requiring users to create their own package definitions.

Hardware support is also limited because Guix uses the Linux-libre kernel, which excludes all proprietary device drivers. This means certain WiFi adapters, graphics cards, and other peripherals may not work out of the box. Furthermore, documentation outside of official sources is sparse. Unlike Ubuntu which has countless community tutorials and forum posts, Guix users often rely primarily on the official manual. Finally, Guix has no graphical configuration tools. Every aspect of system configuration requires editing text files, which can be intimidating for users who prefer graphical administration interfaces. For these reasons, Guix System is best suited for experienced Linux users, developers, and system administrators who value reliability and reproducibility over convenience and hardware compatibility.

Let me know if you want me to rewrite section i (Conclusion) or j (Future outlook) in passage form as well.

i. Conclusion

The installation of GNU Guix System 1.4.0 in VirtualBox was completed successfully, though it required more technical preparation than a typical Linux distribution. The text-based installer and manual partitioning demanded attention to detail, but the result is a uniquely powerful operating system.

Key achievements:

User account created with my full name as specified
Guix System running with Xfce desktop environment

Understanding of declarative configuration through /etc/config.scm

Experience with functional package management concepts

Guix System demonstrates an alternative vision for operating systems – one where reliability, reproducibility, and software freedom take priority over convenience and hardware compatibility. While not suitable for everyone, it represents important innovations in system administration and package management .

j. Future Outlook / Recommendation

Future Outlook:

Guix Home – Declarative user environment management (similar to Nix home-manager) is maturing
Improved image generation – Developers are working to support more filesystems in system images
Potential Hurd kernel support – Guix may eventually run on GNU Hurd, fulfilling a long-term GNU goal
Better ARM/aarch64 support – Growing interest in Guix on Apple Silicon and Raspberry Pi

Recommendations:

Guix System is an excellent choice to demonstrate advanced OS concepts, but allocate 2-3 days for installation and initial configuration.

For beginners: Install Guix as a package manager on your existing Linux distribution first :

bash
wget https://git.savannah.gnu.org/cgit/guix.git/plain/etc/guix-install.sh
chmod +x guix-install.sh
sudo ./guix-install.sh
This lets you learn Guix commands without committing to a full system install.

For the virtual machine:

Use 4+ GB RAM and 25+ GB disk – compilation is resource-intensive
Enable NAT networking to fetch substitutes (pre-built binaries)
Take VirtualBox snapshots before major system reconfigurations

Learning resources:

Official manual: https://guix.gnu.org/manual/
System Crafters YouTube channel – excellent Guix tutorials
IRC: #guix on Libera.Chat

Consider NixOS as an alternative if you find Guix too restrictive – similar concepts but with larger community and more packages .

Virtualization in Modern Operating Systems

What is Virtualization?

Virtualization uses a software layer called a hypervisor (e.g., VirtualBox, KVM, VMware ESXi, Hyper-V) to create and manage one or more virtual machines (VMs). Each VM emulates a complete physical computer with its own virtual CPU, memory, storage devices, and network interfaces, allowing a guest operating system to run isolated from the host operating system.

Why is Virtualization Used?

Isolation – Run untrusted, experimental, or legacy software without risk to the host system
Resource efficiency – Multiple VMs share physical hardware, reducing costs (server consolidation)
Development and testing – Create clean, reproducible environments for software testing
Disaster recovery – VM snapshots allow instant rollback to previous states
Cross-platform compatibility – Run different operating systems on the same physical machine (e.g., Linux on Windows, Windows on macOS)

How Does Virtualization Work in Modern Operating Systems?

Virtualization works by using a software layer called a hypervisor that sits between the hardware and the operating systems. The hypervisor tricks each guest operating system into thinking it has its own complete physical computer, while actually sharing the real hardware among multiple VMs.

Step by step process:

- ✓ Hardware assistance – Modern CPUs (Intel VT-x or AMD-V) have special instructions built directly into the processor that allow the hypervisor to run guest operating systems efficiently. Without these, virtualization would be very slow.

- ✓ The hypervisor takes control – When a virtual machine starts, the hypervisor takes control of the CPU. The hypervisor runs at the most privileged level (called "ring -1" or root mode), which is even higher than a normal operating system's kernel.

- ✓ Trapping and emulating – The guest operating system tries to execute privileged instructions (like accessing hardware directly). The CPU automatically "traps" these instructions and hands them to the hypervisor. The hypervisor then emulates or redirects the request to the real hardware safely.

- ✓ Memory management – The hypervisor uses a technology called Nested Page Tables (NPT) or Extended Page Tables (EPT) . This allows the guest OS to manage its own virtual memory while the hypervisor separately manages how that virtual memory maps to actual physical RAM. This happens automatically and efficiently.

- ✓ Device emulation – For hardware like network cards, disk drives, and USB ports, the hypervisor provides virtual versions. When the guest OS tries to send data to its virtual network card, the hypervisor intercepts it, translates it, and sends it through the host's real network card.

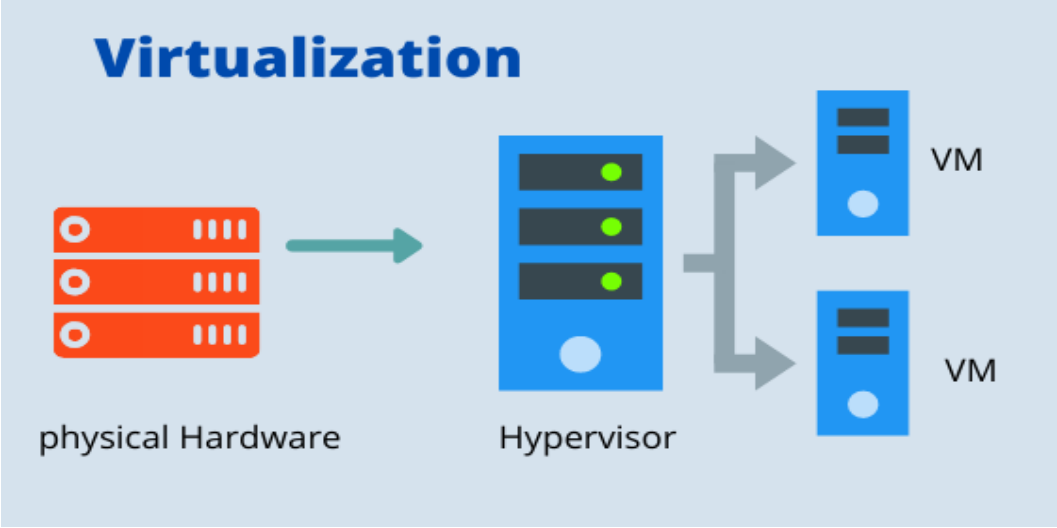
Two types of hypervisors:

Type 1 (Bare-metal) – Runs directly on the hardware with no host OS. Examples: Microsoft Hyper-V, VMware ESXi, KVM (Linux). These are more efficient and used in data centers.

Type 2 (Hosted) – Runs as an application on top of an existing host operating system. Examples: VirtualBox, VMware Workstation. These are common for desktop users.

Simple analogy: Think of the hypervisor as a building manager with many apartments. Each guest operating system is a tenant who thinks they have their own private house with their own kitchen, bathroom, and electricity. In reality, the building manager secretly shares the water heater, electrical wiring, and elevator among all tenants, but each tenant never knows the others exist.

Why this matters – Because of hardware virtualization support in modern CPUs, VMs run at nearly the same speed as the host operating system. The guest OS doesn't know it is virtualized, and the host OS doesn't need to be modified. This is what makes running multiple operating systems simultaneously practical and efficient.



Summery

This project documented the installation of a Linux operating system (GNU Guix System) inside a virtual machine using VirtualBox. The installation process included creating a VM, configuring hardware, partitioning disks, and setting up a user account with full name. Several problems were faced, such as BIOS virtualization being disabled and a package build failure, which were solved by enabling VT-x and increasing VM resources. The installed system uses the ext4 filesystem and offers advantages like system rollbacks but has a steep learning curve. Virtualization was also explained as a technology that allows multiple operating systems to run on one physical machine using a hypervisor and CPU hardware support.